

DS n° 2**LE 13/11/24**

Il est possible dans un exercice d'utiliser les fonctions définies dans les questions précédentes. Hormis celles proposées, on n'introduira pas de bibliothèque supplémentaire.

EXERCICE 1 : AMÉLIORATION DE COMPLEXITÉ**A - Calcul de somme**

On souhaite calculer la somme $S_n = \sum_{k=0}^n k!$

- 1) Proposer une fonction `factorielle(k)` qui prend en argument un entier naturel k et retourne la valeur de $k!$.
- 2) Combien de multiplications sont faites pour l'appel `factorielle(k)` ?

On propose le code suivant :

```
1 def calcul_somme(n):
2     S = 0
3     for k in range(n) :
4         S += factorielle(k)
5     return S
```

- 3) Justifier que la complexité (on se considère que les multiplications) de `calcul_somme(n)` est quadratique.
- 4) Après avoir identifié le point améliorable de la fonction `calcul_somme(n)`, implémenter cette amélioration et montrer que cette nouvelle fonction a un cout linéaire.

B - Puissance rapide

- 5) Proposer une fonction ayant un cout linéaire `puissance(x,n)` prenant en argument un réel x et un entier naturel n , retournant la valeur de x^n .
- 6) Expliquer en quoi la dichotomie permettrait d'améliorer le calcul de x^n .
- 7) Implémenter cette amélioration pour une fonction de cout logarithmique.

C - Recherche de la plus longue sous-chaîne sans caractère répété

Pour une chaîne de caractère donnée, on veut trouver la longueur de la plus longue sous-chaîne d'une chaîne donnée qui ne contient pas de caractères répétés.

Par exemple pour la chaîne " robot" la longueur maximale est de 3, pour "rob".

Voici une proposition

```
1 def naive_longest_unique_substring(s):
2     max_length = 0
3     n = len(s)
4     for i in range(n):
5         seen = []
6         j = i
7         while j < n :
8             if not s[j] in seen:
9                 seen.append(s[j])
10                j += 1
11            else :
12                j = n
13        max_length = max(max_length, len(seen))
14    return max_length
```

- 8) Parcourir pas à pas cette fonction pour `s="robot"`.
- 9) En admettant que ligne 8 utilise une recherche séquentielle linéaire, donner la complexité de cette fonction dans le pire cas, en précisant le pire cas possible.

La **fenêtre glissante** (ou *sliding window*) est une technique utilisée pour parcourir des sous-parties contiguës d'une structure linéaire (comme un tableau ou une chaîne de caractères) tout en réduisant les calculs redondants.

IDÉE GÉNÉRALE

- **Définir une "fenêtre"** : On maintient une section contiguë de la structure à examiner (par exemple, une plage d'indices [start,end]).
 - **Avancer la fenêtre progressivement** :
 - ◊ On étend l'extrémité droite (end) pour inclure de nouveaux éléments.
 - ◊ On contracte l'extrémité gauche (start) si nécessaire, par exemple lorsqu'une contrainte n'est plus respectée (comme éviter les caractères répétés).
 - **Effectuer des calculs partiels** : On utilise les informations accumulées dans la fenêtre pour calculer efficacement des résultats locaux, évitant ainsi de recalculer chaque sous-ensemble depuis zéro.
- 10) Appliquer cette idée au problème pour proposer une fonction dont la complexité est améliorée. On justifiera cette amélioration.

EXERCICE 2 : PERMUTATIONS DE LISTES

DÉFINITION 1

Soit $n \in \mathbb{N}$. Une permutation de l'ensemble $\llbracket 0, n-1 \rrbracket$ est une bijection de $\llbracket 0, n-1 \rrbracket$ sur $\llbracket 0, n-1 \rrbracket$.

On représente en Python une permutation par une liste, dont l'élément d'indice i est l'image de i par cette permutation. Par exemple, pour $n = 4$ la liste $[3, 1, 0, 2]$ représente la permutation p définie par $p(0) = 3$, $p(1) = 1$, $p(2) = 0$ et $p(3) = 2$.

A - Exemples

- 1) Si L est une liste Python représentant une permutation pour $n = 4$ quelle instruction Python permet de trouver l'image de 1 par cette permutation?
- 2) Pour $n = 5$ quelle liste Python représente la permutation p telle que $p(0) = 1$, $p(1) = 3$, $p(2) = 2$, $p(3) = 0$ et $p(4) = 4$?

B - Support

- 3) Implémentez une fonction `support(p)` qui renvoie l'ensemble des indices où les éléments d'une permutation p ne sont pas à leur position d'origine.

• Exemple :

```
1 >>> support([2, 1, 0])
2 [0, 2]
3 >>> support([1, 2, 0])
4 [0, 1, 2]
```

- 4) Pour $n = 5$ donner un exemple de permutation telle que `support(p) = [1, 3, 4]`.

C - Composition des permutations

- 5) Implémentez une fonction `compose(p, q)` qui calcule la composition de deux permutations p et q , définie par $r[i] = q[p[i]]$ (attention à l'ordre).

• Exemple :

```
1 >>> compose([2, 0, 1], [1, 2, 0])
2 [0, 1, 2]
```

- 6) Quelle commande (d'une seule ligne) permettrait, pour deux trois permutations données, de voir que la composition est associative, c'est à dire :

$$\text{compose}(\text{compose}(p, q), r) = \text{compose}(p, \text{compose}(q, r)).$$

D - Inverse

Une permutation est bijective, il existe donc une permutation inverse.

7) Implémentez une fonction `inverse(p)` qui retourne l'inverse d'une permutation p .

- **Exemple :**

```
1 >>> inverse([2, 0, 1])
2 [1, 2, 0]
```

8) Quelle commande (d'une seule ligne) permettrait de vérifier que votre est bon pour une permutation p donnée?

E - Génération de permutations

9) Donner toutes les permutations possibles de $\llbracket 0, 2 \rrbracket$

10) Écrivez une fonction récursive `generate_permutations(lst)` qui prend en entrée une liste d'entiers et renvoie la liste de toutes les permutations possibles de cette liste.

- **Exemple :**

```
1 >>> generate_permutations([0, 1])
2 [[0, 1], [1, 0]]
```